Building a Volatility-Aware Options Screener: Research, Design, and Implementation

Kavin
Independent Project

---

1. Why I Built This

I've spent the last year teaching myself to turn questions into code. First with a research paper on market patterns, then with trading indicator work, and now with this project. Options intrigue me due to the many factors that influence pricing. Many contracts look enticing if you ignore liquidity, execution costs, or the difference between a fair bet and a good one. So I wanted to explore all of this, learn about the factors that influence option pricing, and create something out of it. And so I created a dashboard that accounts for those factors and ranks option strategies.

Before I started any code, a lot of learning was required. I spent a lot of time reading information about options, starting with the basic Greeks and moving into more advanced models and factors like Black-Scholes and binomial pricing, Heston and SVI volatility surfaces, and the variance risk premium literature (Corsi's HAR-RV model, Bollerslev-Tauchen-Zhou). It was only after this that I turned that research into a concrete plan and started building.

2. The Research Phase

The core idea I took from the research is that there's no single formula that reliably finds a mispriced option. Whatever real edge exists for a personal screener tends to come from one of a few places: noticing when the market's implied volatility disagrees with what you'd actually forecast realized volatility to be, calculating a trade's true expected value across every possible outcome instead of just the best and worst case, or finding contracts that look rich or cheap compared to a fitted curve of the whole volatility surface. That third one is easy to fool yourself with. A messy quote, a thin market, or a wrong assumption about dividends can make something look "mispriced" when it's really just an error in your own model. So I built the safer version first: pair a volatility read with a proper expected-value calculation, and treat the option Greeks as a check on risk, not as a source of edge on their own. That one decision shaped everything else I built.

3. What I Actually Built

The project is a full working app, containing a Python backend (using FastAPI), a small database to store data, and a React frontend for the dashboard itself. Data comes from Massive.com's options API, which gives live option chain data (bid/ask prices, volume, open interest, implied volatility, and the Greeks), but it doesn't hand you a ready-made history of past implied volatility. So the app is built to save its own daily snapshots over time and build that history itself.

The backend is roughly 4,300 lines of Python, split into a handful of pieces that each do one job:

Liquidity gate - Before any contract is even considered for ranking, it's checked for how easy it would actually be to trade: how wide the bid/ask spread is, how much open interest and volume it has. This exists so the app never tells you a contract looks like a "great trade" when in reality nobody's actually trading it and you'd get a bad price the moment you tried.

IV Rank and IV Percentile - These take today's implied volatility and put it in context against the last year of history, so you can tell whether current volatility is unusually high, unusually low, or normal. If there isn't enough saved history yet, the app says so directly instead of pretending to be confident. The app also calculates its own forecast of how volatile the stock is likely to be, based on its recent price history, and compares that forecast to the market's implied volatility. The gap between the two is the whole point, it tells you whether the market is pricing in more movement than you'd expect, or less. From there, a strategy builder puts together actual tradeable setups: credit spreads, debit spreads, and iron condors (a structure with four separate legs). Each leg is picked based on how far out of the money it is, and every single leg has to pass the liquidity gate first before it's allowed into a candidate trade.

Expected value / probability engine - A common shortcut in retail tools is to calculate a trade's expected value as a simple bet: chance of winning times the max win, minus chance of losing times the max loss. That's actually wrong for spreads and condors, because the stock can land anywhere, and most outcomes are partial wins or partial losses, not just the two extremes. My version instead lays out hundreds of possible prices the stock could be at when the option expires, calculates the exact profit or loss at every single one of those prices, and weighs each one by how likely it actually is. It does this twice: once using my own volatility forecast, and once using the market's implied volatility, so the dashboard can show both "what I expect" and "what the market is pricing in" side by side, and let you see when they disagree.

Greek efficiency check - The Greeks for a whole strategy (not just one leg) get added up and turned into a score that rewards a strategy for collecting good return without taking on too much unwanted risk, rather than just rewarding whichever strategy has the single biggest number, which usually just points you toward the riskiest, shortest-dated contracts.

4. The Liquidity Gate

The liquidity gate runs before anything else, because a good-looking score on an untradeable contract is worse than no score at all. Contracts that fail the hard filters are removed entirely; contracts that pass but look weak keep a warning attached and score lower. Table 1 lists the default thresholds.

Table 1. Default liquidity thresholds. All are configurable at runtime.

Filter	Default	If violated
Open interest + volume	≥ 100 combined	Rejected
Bid	> 0	Rejected
Ask	> bid	Rejected
Bid/ask spread	≤ 15% of mid	Rejected

Filter	Default	If violated
Spread, far out-of-the-money	≤ 25% of mid	Allowed, penalized
Quote age, under 7 DTE	≤ 45 minutes	Rejected
Quote age, 14+ DTE	≤ 120 minutes	Warning only

The liquidity score itself is a weighted blend of those same inputs. When live bid/ask data is available, the spread carries 50% of the weight, open interest 30%, and volume 20%. When the provider omits quotes, which happens regularly on the delayed tier, the app falls back to weighting open interest at 55% and volume at 35%, and treats the missing spread as neutral rather than silently assuming it is fine. I added that fallback after finding the original filter rejecting heavily traded near-the-money contracts purely because the provider had returned no quote for them.

5. Volatility and Expected-Value Model

The volatility read has two halves. The first is a forecast of how much the stock will actually move, built by blending realized volatility over three lookback windows, weighted toward the most recent: 50% on the last 10 days, 30% on the last 20, and 20% on the last 60. Blending several windows matters because volatility clusters at more than one timescale, so a single lookback either overreacts to a quiet stretch or lags a real change in regime.

The second half compares that forecast against the market's implied volatility. The difference is computed in variance space rather than volatility space, which is the convention in the academic literature, then converted to a 0–100 score. A large positive gap means the market is pricing in more movement than the forecast expects, which favors selling premium; a large negative gap favors buying it.

The expected-value engine then evaluates each candidate across a grid of 800 possible expiration prices, spanning roughly four standard deviations in each direction. Each price gets an exact payoff and a lognormal probability, and the probabilities are normalized to sum to one so the result is a valid distribution rather than a raw density. Commission ($0.65 per contract) and slippage ($0.02 per contract) are subtracted before scoring, not after, so no candidate can rank well on an edge that execution costs would erase. The whole calculation runs twice, once under the forecast volatility and once under the market's implied volatility, and both results are displayed.

6. Composite Scoring

Each component score is on a 0–100 scale, then combined using weights that depend on the type of strategy being evaluated. Table 2 gives the three profiles.

Table 2. Composite score weights by strategy profile.

Component	Short premium	Long volatility	Neutral
Volatility edge	0.25	0.25	0.20
Expected value (Alpha)	0.25	0.25	0.30

Component	Short premium	Long volatility	Neutral
Liquidity	0.20	0.20	0.25
Greek efficiency	0.15	0.15	0.15
IV Rank / Percentile context	0.10	—	—
Risk / reward	0.05	0.05	0.10

Alpha is the expected value divided by maximum loss, which puts trades risking very different dollar amounts on a comparable scale. It is mapped onto the 0–100 range so that an Alpha of 0.02 scores 60, 0.05 scores 75, and 0.10 or better saturates at 100.

Penalties are then subtracted from the weighted total. These are deliberately blunt, because each one represents a way the underlying model becomes less trustworthy, not merely a less attractive trade.

Table 3. Risk penalties subtracted from the composite score.

Condition	Penalty
Fewer than 7 days to expiration	−30
Negative modeled expected value	−12
Fewer than 14 days to expiration	−10
High probability of profit with negative expected value	−8
Short call with assignment / ex-dividend exposure	−5
Delayed provider data in use	−3
IV history still accumulating	−3

The short-dated penalty is the largest for a specific reason: the lognormal probability model the expected-value engine relies on degrades badly as expiration approaches, so a high score on a contract expiring in three days is more likely to reflect model error than genuine opportunity. For the same reason, IV Rank and IV Percentile are held at a neutral value until at least 30 daily snapshots have been stored, rather than being computed from a handful of days and presented as though they carried a year of context.

Final scores are graded A at 90 and above, B at 80, C at 70, D at 60, and F below that. Every candidate also carries a written explanation assembled from its own score breakdown, and every explanation ends with the same two statements: that the result requires backtest confirmation, and that it is not financial advice.

7. Testing

I tried to treat this less like a quick trading script and more like a small, careful codebase. There are 70 automated tests across 13 files, roughly 990 lines of test code against 4,300 lines of application code, and all 70 currently pass. The distribution is deliberately weighted toward the math rather than the plumbing.

Table 4. Automated test coverage by module.

Module under test	Tests	Representative assertion
Liquidity filtering	10	A contract with no bid is rejected outright
IV Rank / Percentile	8	Current IV at the historical minimum returns 0
Candidate tracking	8	Stored candidates reprice against later quotes
Payoff calculations	7	Iron condor payoff correct at every terminal price
Probability grid	7	Probabilities sum to 1 across the grid
Realized volatility	7	A constant price series produces zero volatility
Composite scoring	6	A risk penalty measurably lowers rank
Strategy builder	4	No candidate uses a leg that failed liquidity
Contract metrics	4	Mid, spread, and DTE derived correctly
Schemas, chain, health, normalization	9	Malformed provider fields do not crash the app

The test for realized volatility on a flat price series and the test that probabilities sum to one are both trivial to write and were both worth writing. Neither failure would have been visible from looking at dashboard output, since a subtly wrong volatility number still renders as a perfectly plausible-looking score.

The code that talks to Massive's API also has real retry logic with exponential backoff, and it translates the provider's specific data format into the app's own internal format, so the rest of the app never has to know or care exactly how Massive structures its data. That means I could swap in a different data provider later without having to touch any of the scoring logic.

9. What I Took Away From It

The screener works. It pulls option chains, filters out contracts that cannot realistically be traded, reads the volatility regime, builds credit spreads, debit spreads, and iron condors from the surviving contracts, evaluates each one across the full range of possible outcomes, and ranks the results with a written explanation attached to every candidate. The analytics are covered by 70 automated tests, and the provider layer is separated from the model layer, so the data source can change without the scoring logic changing with it.

The next step is backtesting. The app already stores every chain snapshot and every score it produces, which is the dataset a historical replay needs, so the groundwork is in place. Until that replay exists, the scoring model is a well-specified hypothesis rather than a validated one, and the dashboard is written to say exactly that.

What I most want to keep from this project is the order I did things in. I read the research before I designed the system, and I designed the system before I wrote the code. That order is why the app rejects bad contracts before it scores them instead of after, why expected value is computed across every

outcome instead of two, and why the parts that could quietly produce wrong answers are the parts with the most tests behind them. The screener is the artifact, but the sequence is the part I would repeat.